

Agentic AI Engineering Internship 2026

Week 1 Comprehensive Report

AI Fundamentals & Agentic AI Foundations

Prepared By:

Ghafoor

Date:

June 30, 2026

CalderR.

RETHINKING · HOW · WORK · WORKS

Contents

1	Executive Summary	2
2	Project Deliverables	2
2.1	Production Project: AI-Powered Customer Support Platform	2
2.1.1	Architecture & Tech Stack	2
2.1.2	Key Features & Implementation Details	2
2.2	Intermediate Project: Intelligent CLI Assistant	3
2.2.1	Key Features & Implementation Details	3
3	Foundational Labs	4
3.1	Lab 1.1: Groq CLI Chatbot	4
3.2	Lab 1.2: Manual ReAct Agent Loop	4
3.3	Lab 1.3: Prompt Engineering Evaluator	4
4	Conclusion & Next Steps	4

1 Executive Summary

This report outlines the technical achievements and learning outcomes completed during Week 1 of the CalderR Agentic AI Engineering Internship. The core objective of the week was to establish a robust foundation in Large Language Models (LLMs) and transition from traditional reactive AI paradigms to proactive, agentic systems.

Over the course of the week, three foundational labs were completed, focusing on manual conversation state management, building ReAct (Reasoning and Acting) loops from scratch, and systematically evaluating prompt engineering strategies. The week culminated in the successful deployment of two comprehensive projects: an **Intelligent CLI Assistant** (Intermediate) and an **AI-Powered Customer Support Platform** (Production). Both projects demonstrate a strong mastery of LangChain, the Groq API, conversational memory management, and modern software engineering practices.

2 Project Deliverables

This section provides a detailed architectural and functional breakdown of the two major projects built this week.

2.1 Production Project: AI-Powered Customer Support Platform

Project Overview

A production-ready, full-stack customer support chatbot featuring an asynchronous FastAPI backend, persistent SQLite memory, robust rate-limiting, and a modern React frontend with glassmorphism UI design. Fully containerized using Docker.

2.1.1 Architecture & Tech Stack

The platform operates on a decoupled client-server architecture, ensuring high scalability and clean separation of concerns:

- **Frontend:** React, Vite, Vanilla CSS (Custom Glassmorphism aesthetics).
- **Backend:** FastAPI (Python), Uvicorn.
- **AI/LLM Core:** LangChain, Groq API (llama-3.1-8b-instant).
- **Database & Memory:** SQLite, SQLChatMessageHistory.
- **Infrastructure:** Docker, Docker Compose.

2.1.2 Key Features & Implementation Details

1. **Asynchronous REST API:** The FastAPI backend exposes highly efficient endpoints (`/sessions`, `/messages`, `/health`). Data validation is strictly enforced using Pydantic models.

2. **Persistent Conversational Memory:** Instead of relying on volatile in-memory buffers, the system utilizes LangChain's `SQLChatMessageHistory` to persist conversation turns to a local SQLite database (`chat_history.db`). This allows users to retain context across disconnected sessions seamlessly.
3. **Production Safeguards (Middleware):**
 - **Rate Limiting:** Implemented a custom middleware that restricts IP addresses to 20 requests per 60-second sliding window, returning HTTP 429 upon violation.
 - **Logging:** Comprehensive request timing, method, and status code logging via custom HTTP middleware ensures observability.
 - **CORS:** Properly configured Cross-Origin Resource Sharing to securely allow frontend-backend communication.
4. **Containerization:** The entire stack is orchestrated via `docker-compose.yml`, allowing for a seamless, one-click spin-up environment spanning both the frontend client and the backend server.

2.2 Intermediate Project: Intelligent CLI Assistant

Project Overview

A highly interactive, terminal-based AI assistant strictly scoped to the domains of Programming and Software Engineering. It leverages LangChain and the Rich library to deliver beautifully formatted markdown directly in the CLI.

2.2.1 Key Features & Implementation Details

- **Domain-Specific Guardrails:** The assistant uses targeted system prompts to constrain discussions. If queried about non-technical topics (e.g., cooking), the assistant gracefully steers the user back to software development topics.
- **Stateful Memory Management:** Utilizes LangChain's `RunnableWithMessageHistory` to wrap a `ChatGroq` model. The assistant maintains a continuous short-term memory buffer within a given session, allowing it to accurately answer follow-up questions (e.g., "optimize that code using memoization").
- **Premium Terminal UI (Rich):** The user experience is elevated by formatting model outputs dynamically using the `rich.markdown` and `rich.panel` modules, complete with syntax-highlighted code blocks, animated "thinking" spinners, and stylized user/assistant tags.
- **Command Interception:** Implemented a custom input loop that successfully captures and executes CLI commands such as `/clear` (to wipe session history) and `/exit`.

3 Foundational Labs

3.1 Lab 1.1: Groq CLI Chatbot

Constructed a base terminal chatbot integrating directly with the Groq Python SDK.

- **Memory Management:** Manually handled state by dynamically appending `user` and `assistant` role dictionaries to an evolving messages list.
- **Token Analytics:** Implemented a custom `TokenTracker` class to monitor prompt, completion, and total token usage per turn, explicitly tracking the exponential growth of context windows.
- **Model Switcher Enhancement:** Exceeded base requirements by adding a real-time `/model` selection feature, allowing hot-swapping between models (e.g., Qwen 3.6 - 27B) mid-conversation without dropping context.

3.2 Lab 1.2: Manual ReAct Agent Loop

Developed an autonomous agent from scratch without relying on orchestration frameworks like LangChain or AutoGen.

- **Cognitive Loop:** Implemented the fundamental `Thought` → `Action` → `Action Input` → `Observation` loop parsing raw text generated by the LLM.
- **Tool Execution:** Built and integrated custom tools including a Fact Search against a mock knowledge base and a secure mathematical calculation sandbox utilizing a restricted `eval()` whitelist to prevent arbitrary code execution.
- **Safeguards:** Implemented a hard limit (`MAX_ITERATIONS` = 6) to prevent infinite reasoning loops.

3.3 Lab 1.3: Prompt Engineering Evaluator

Conducted a systematic A/B test evaluating five distinct system prompts on a news summarization task.

- **Findings:** Discovered that *Chain of Thought* prompts yielded the highest accuracy by forcing the extraction of statistics before summarization. Conversely, strict format constraints (e.g., "Exactly 20 words") significantly damaged factual accuracy in favor of structural conciseness.
- **Persona Implementation:** Demonstrated that adopting specialized personas heavily influenced output tone and density without requiring changes to the underlying model logic.

4 Conclusion & Next Steps

Week 1 successfully established the core building blocks required for advanced agentic workflows. By building systems manually before relying on abstractions like LangChain,

a deep understanding of token usage, state management, and the ReAct architecture was achieved. Both projects met all required specifications, featuring strict type-hinting, environment variable security, and comprehensive documentation.

Moving into Week 2, the focus will shift toward scaling these architectures with external memory (Vector Databases/RAG) and more complex multi-agent orchestration.